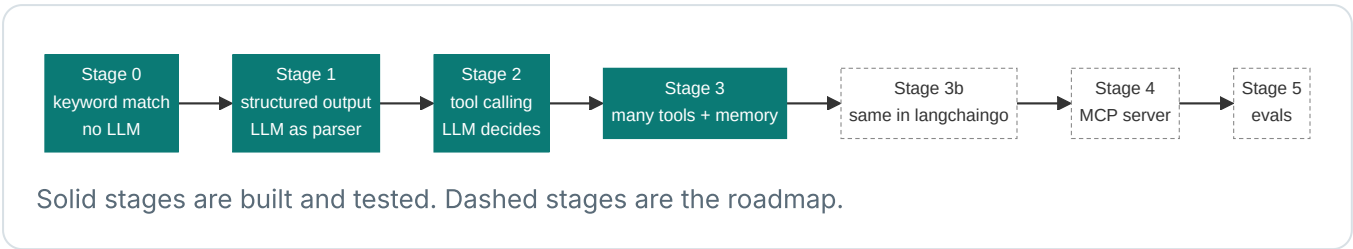


Build an agent from scratch, one idea at a time

agent-lab is a small Go project that asks for a joke in plain English and fetches one from a real API. The domain is tiny on purpose, so each stage can teach a single idea about LLM apps without anything else in the way.



Who is in charge at each stage?

The whole project is really one dial: how much control moves from your code to the model.

STAGE	INTERPRETS THE REQUEST	CHOOSES THE NEXT ACTION	RETRIES ON FAILURE
0 Keywords	your code	your code	☐
1 Structured	the model	your code	☐
2 Tool loop	the model	the model	☒
3 Multi-tool	the model	the model	☒

The one rule that runs through every stage

DESIGN RULE

The joke client takes a typed `jokeapi.Request`, never free text.

The predecessor project had the model write `category=misc&blacklist=racist` as a string, and a keyword matcher re-parsed it. The blacklist was silently dropped. Here the model fills in a typed form, our code runs it through `Normalize()`, and only then does anything touch the network.

Repository map

<code>cmd/jokes/</code>	REPL; picks a stage with <code>-stage</code>
<code>internal/jokeapi/</code>	typed JokeAPI client + the Request schema
<code>internal/dadjoke/</code>	icanhazdadjoke.com client (second joke source)
<code>internal/ollama/</code>	raw-HTTP Ollama chat client with memory limits
<code>internal/stage0/</code>	keyword interpreter
<code>internal/stage1/</code>	structured-output interpreter (<code>ollama.go</code> , <code>claude.go</code>)
<code>internal/stage2/</code>	tool-calling agent loop
<code>internal/stage3/</code>	multi-tool agent with memory (<code>tools.go</code> = tool registry)
<code>internal/trace/</code>	step printer behind <code>-trace</code>

Plain HTTP, no SDK

The Ollama client is `net/http` plus structs that mirror the wire format. Reading `internal/ollama` shows exactly what a tool call looks like on the wire.

Duplication is intentional

Stages are meant to be read side by side, so the agent loop is copied between stage 2 and stage 3 rather than shared.

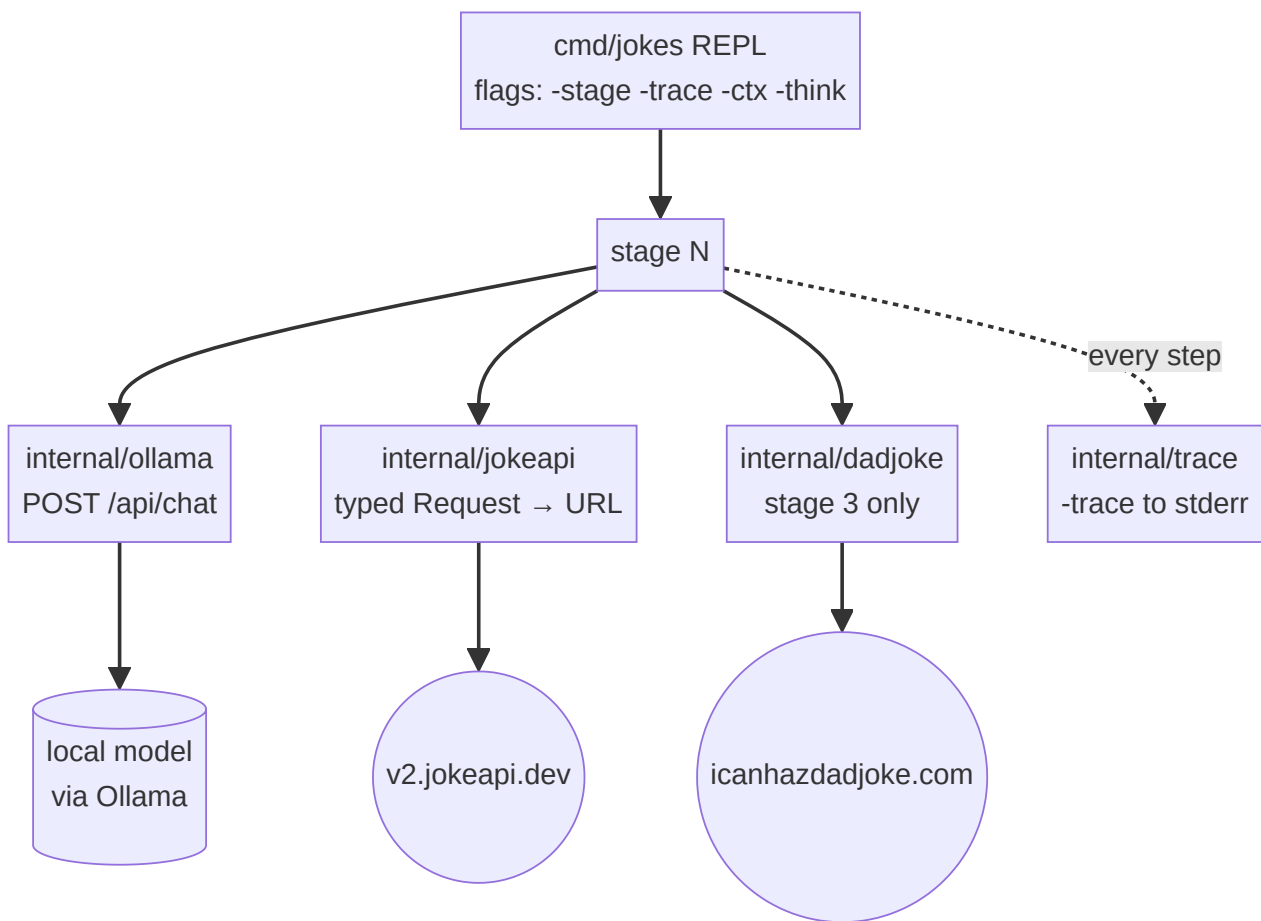
Everything is tested offline

Every API and model is faked with `httptest`. `make test` needs no network. `make live` hits the real joke APIs.

Local and frugal

Default model `qwen3.5:9b-mlx` through Ollama on a 16 GB Mac, with context 8192, output cap 2048 and a 2 minute keep-alive.

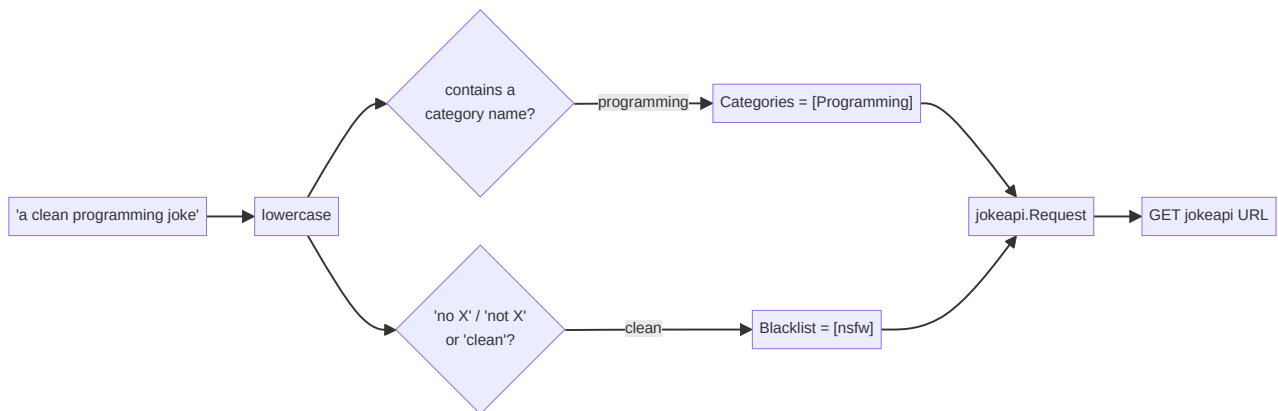
The shared plumbing



The REPL builds one stage, and every stage reuses the same three clients and the tracer.

Keywords: the baseline to beat

Before adding an LLM, solve the problem without one. `stage0.Interpreter` lowercases your text and looks for known words: category names, "twopart" or "one-liner", and the pattern `no <flag>` or `not <flag>`.



Everything here is `strings.Contains`. It is fast, free and predictable.

Where it works

"programming joke", "a two-part pun", "no politics". Exact words in, exact filters out.

Where it breaks

"keep it family friendly" matches nothing. "clean" maps only to `nsfw`, not the full blacklist. "nothing about religion" slips past `no religious`.

LESSON

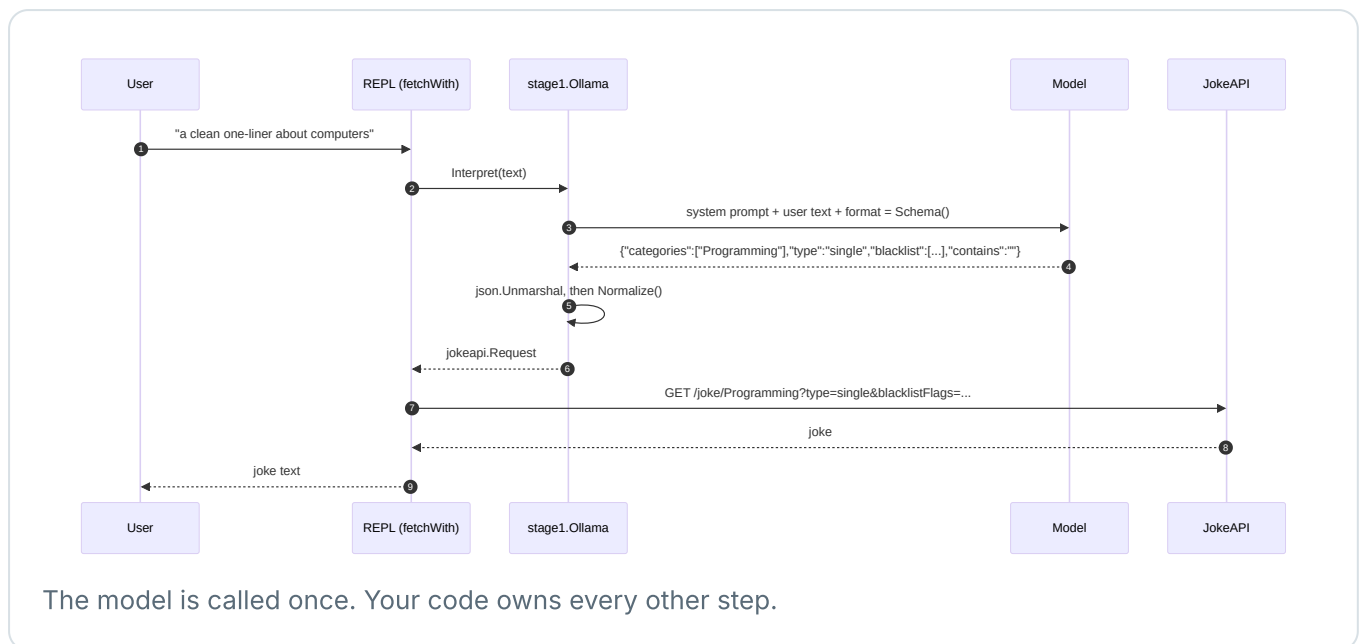
A baseline makes every later stage measurable. If the LLM can't beat `strings.Contains`, it isn't earning its cost.

TRY IT

```
go run ./cmd/jokes -stage 0 -trace
> keep it family friendly      # watch the empty blacklist in the trace
```

Structured output: the model as a parser

One LLM call, and the model's only job is to fill in a form. We send a JSON schema in Ollama's `format` field. Ollama compiles it into a grammar, so the model can only emit tokens that keep the JSON valid. Our code decodes the reply, calls `Normalize()`, and does everything else itself.



The schema is the contract

`jokeapi.Schema()` lists every property as required and forbids extras, which strict structured-output modes demand. "No preference" is spelled `"any"`, `[]` or `""`.

```
// what the model must produce
{
  "categories": ["Programming"], // enum of 6 categories
  "type":       "single",        // "any" | "single" | "twopart"
  "blacklist":  ["nsfw", "racist"], // enum of 6 flags
  "contains":   ""               // literal substring, usually empty
}
```

Why `Normalize()` still exists

Even under a schema, small models produce casing differences, duplicates and `"any"`.

`Normalize()` matches values case-insensitively against the known lists, drops

unknowns and duplicates, and maps "any" to empty. Model output is data to be cleaned, never trusted input.

Two backends, one prompt

Ollama (`ollama.go`)

Local model. Thinking is turned off because a form-filling task doesn't need it and reasoning costs seconds. Fails loudly if `done_reason` is `length`.

Claude API (`claude.go`)

Same system prompt and schema through the Anthropic SDK, for side-by-side comparison. Needs a Console API key, which a Pro subscription doesn't include.

LESSON

With a schema, the model cannot return anything your code can't decode. The LLM becomes a typed function: text in, struct out.

Known weakness

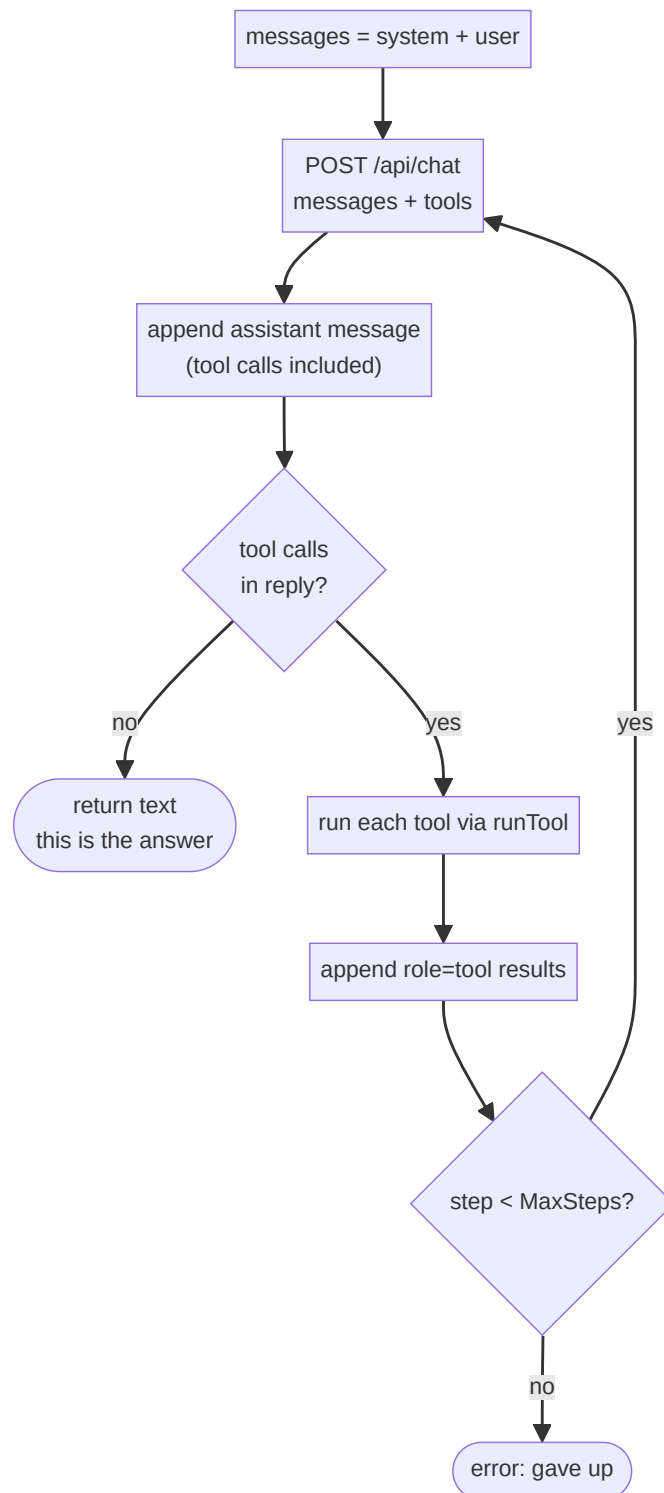
"a joke about the chicago cubs, it can be dirty but not racist" becomes `contains: "cubs"`, which matches nothing, and stage 1 has no way to recover. That is the opening for stage 2.

TRY IT

```
go run ./cmd/jokes -stage 1 -trace
> nothing about politics or religion    # compare with -stage 0
```

Tool calling: the agent loop

In stage 1 your code decided what happens after the model answers. Now the model decides. We describe a `get_joke` tool, and the model chooses when to call it, with what arguments, and what to do with the result. The whole agent is one `for` loop in `Respond`.



Frameworks and SDK tool runners are conveniences over exactly this loop.

A tool has two halves

What the model sees

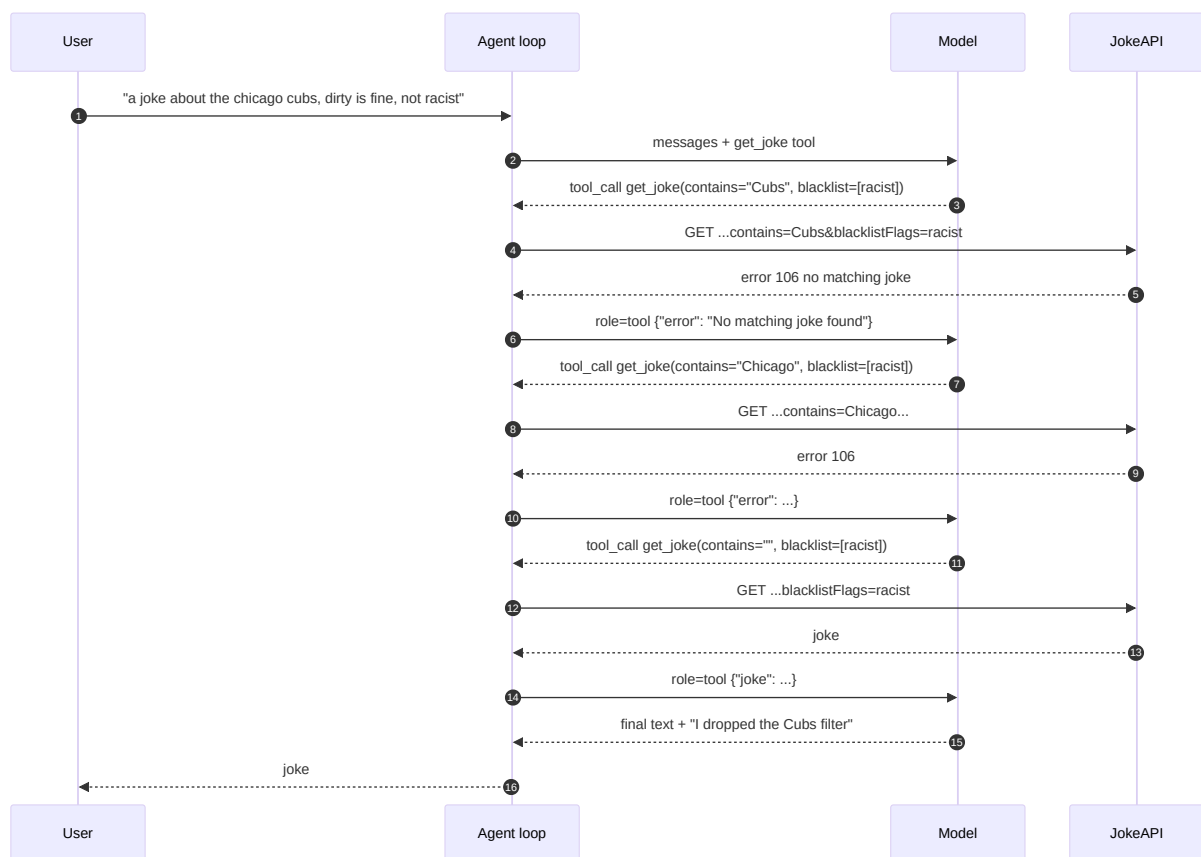
A name, a natural-language description and a JSON schema for parameters. Stage 2 reuses `jokeapi.Schema()` as the parameter schema, so structured output and tool input share one definition.

What your code does

`runTool` decodes the arguments, runs `Normalize()`, calls `Fetch`, and returns JSON text. Unknown tool names and bad arguments also come back as text.

Errors are results, not exceptions

If JokeAPI says "No matching joke found", the loop does not abort. The error goes back as a tool message, and the model reads it and tries again. That is how retry behaviour appears without writing a retry loop.



The retry sequence from the README. Notice `racist` stays on the blacklist through every attempt.

Guard rails in the prompt and in code

GUARD	WHERE	WHY
MaxSteps = 6	code	A confused model could loop on tool calls forever, and each step costs seconds locally.
"Never loosen the blacklist"	system prompt	Retrying is allowed, weakening the user's safety limits is not.
"After 3 failed calls, give up"	system prompt	Gives the model a graceful way out before MaxSteps hits.
done_reason = length	code	A truncated reply is reported as an error, not shown as an answer.
<code>-think</code> flag	code	Optional reasoning before acting. Smarter, slower. Compare the step counts.

LESSON

An agent is a loop where the model's output can be an action. The assistant message with its tool calls must go back into history, or the model forgets what it already asked for.

TRY IT

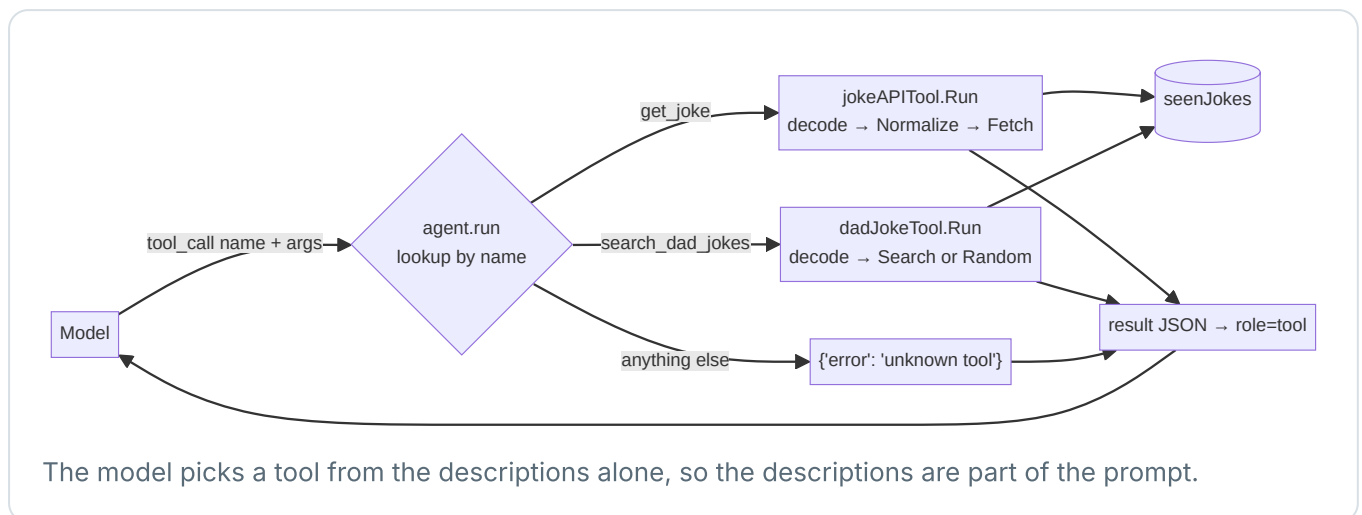
```
go run ./cmd/jokes -stage 2 -trace
> a joke about the chicago cubs, it can be dirty but not racist
# then repeat with -think and compare the number of steps
```

Many tools, memory and a limited context window

Stage 3 adds three things at once. They belong together because each one raises a question the next one answers.

1. A tool registry and tool choice

There are now two tools. `get_joke` wraps JokeAPI and supports categories and content blacklists. `search_dad_jokes` wraps icanhazdadjoke and takes one search word. Each is a `Tool{Def, Run} : Def` is what the model sees, `Run` is what your code does. The agent keeps a map from name to tool.



2. Memory is re-sending the transcript

The model has no memory. The agent stores `history` (without the system prompt) and sends it again on every call. That single change makes "another one", "explain that one" and "keep it clean from now on" work. Some turns need no tool at all, and the model has to notice that.

Memory in the model

The conversation text. It is flexible but fuzzy, and it is the first thing lost when trimmed.

Memory in your code

`seenJokes` records which joke IDs were already told and makes tools re-fetch (up to 3 times) on a repeat. It is exact and survives trimming.

3. Trimming the context window

History grows every turn but `num_ctx` stays at 8192. Before each user turn, `trim()` drops the oldest whole turn while either condition holds:

- the number of turns has reached `MaxTurns` (10), or
- the last prompt used more than 3/4 of the context window ($\text{NumCtx} * 3 / 4$, so 6144 tokens).

It cuts whole turns only. Dropping half a turn would leave a tool result with no call to answer.

Before trim



After trim

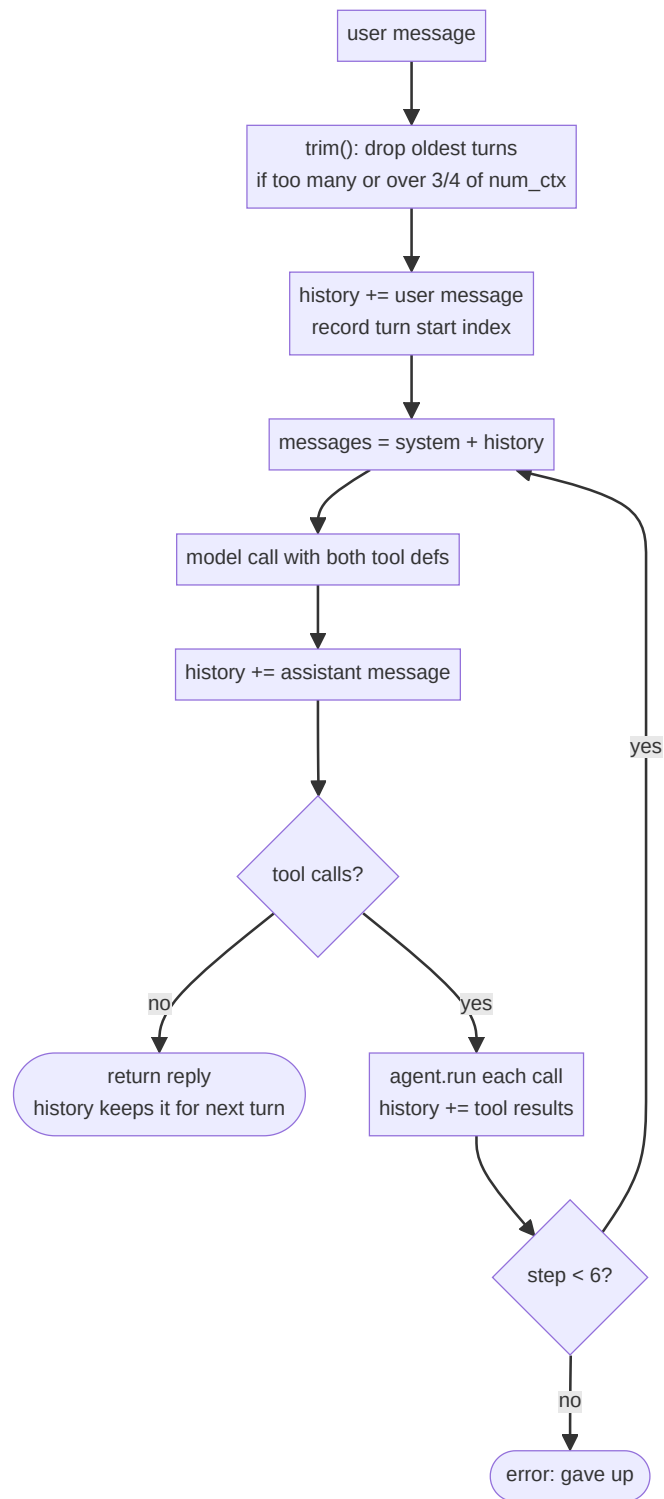


The restriction the user gave in turn 1 disappears with the turn. The model now has no idea it ever existed.

The trade-off this stage exists to show

A restriction given early ("nothing racist") can be trimmed away. Real systems summarize old turns or pin standing instructions in the system prompt. Run with `-ctx 2048` to force trimming early and watch it happen. A second known weakness: after an "explain it" turn, the model sometimes explains later jokes unasked, and sometimes wrongly.

One user message, end to end



Same loop as stage 2. The differences are where history lives and the trim step in front.

A conversation to try

```
go run ./cmd/jokes -stage 3 -trace
> a joke about dogs           # likely search_dad_jokes("dog")
> another one                 # memory: same tool, new joke
> explain that one            # no tool needed
> from now on nothing political. a programming joke # likely get_joke, blacklist=[
> /reset                      # forget history and seenJokes
```

LESSON

Tools, memory and context budget are one system. Whatever you cannot afford to lose should live in code or in the system prompt, not in the oldest message.

What comes next

These stages are not built yet. This section describes the plan from the README and what each one should teach.

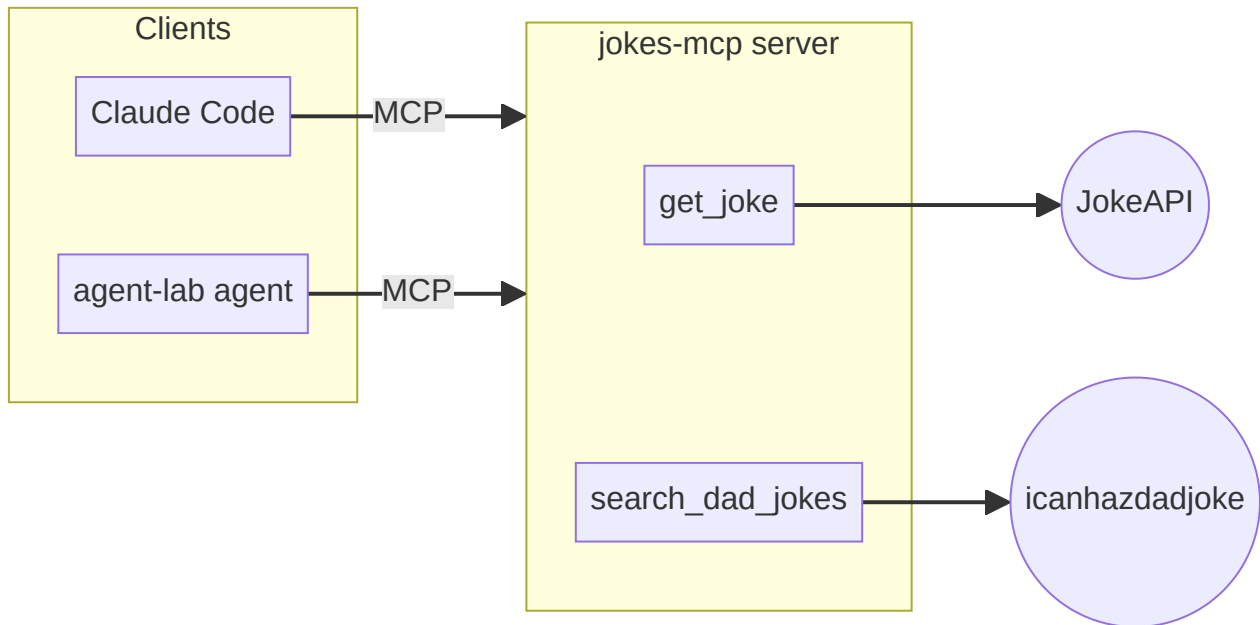
Stage 3b: the same agent in langchaingo

Rebuild stage 3 with a framework, and read the difference. The point is to see what the framework hides. Each hand-written piece has a counterpart:

HAND-WRITTEN IN STAGE 3	FRAMEWORK CONCEPT	WHAT TO LOOK FOR
Ollama HTTP structs	An LLM provider object	Which fields (<code>num_ctx</code> , <code>keep_alive</code>) are still reachable?
<code>Tool{Def, Run}</code> registry	A tool interface with name, description and a call method	Who writes the JSON schema, and can you still reuse <code>jokeapi.Schema()</code> ?
<code>for step := ...</code> loop	An agent executor	Where is the step cap? What does it do on a tool error?
<code>history</code> + <code>trim()</code>	A memory component	Is trimming turn-aware, and does it protect early restrictions?
<code>-trace</code>	Callbacks	Can you still see every prompt and tool result?

Stage 4: an MCP server for the joke tools

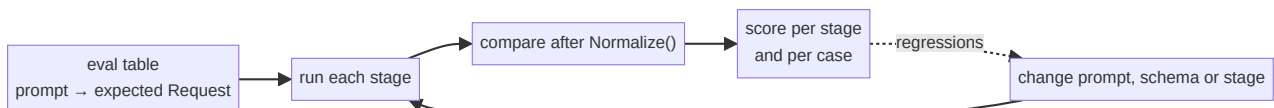
Model Context Protocol moves the tools out of the agent process. The same two tools become a server (`cmd/jokes-mcp`) that any MCP client can use, including Claude Code. The agent no longer owns its tools, it discovers them.



Planned shape. One tool implementation, many clients.

Stage 5: evals

A table of prompts mapped to the expected `jokeapi.Request`, scored against each stage. The two known weaknesses are ready-made cases: "clean" not expanding to a full blacklist in stage 1, and stage 3 explaining later jokes after an "explain it" turn.



Evals turn "it seems better" into a number you can track.

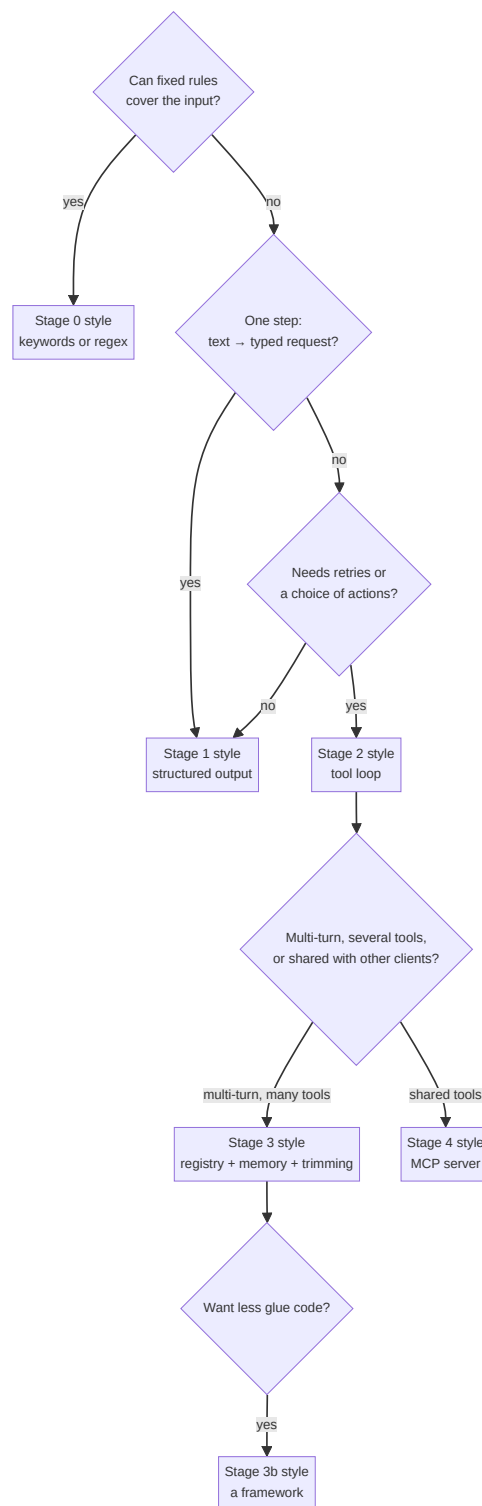
After that: provider switching (Claude, OpenAI, Ollama) behind a single interface.

Applying this to other projects

Jokes are a stand-in. The structure carries over to anything where a person asks in plain language and a typed API does the work. Swap the domain and keep the skeleton.

IN AGENT-LAB	IN YOUR PROJECT	EXAMPLE: SUPPORT-TICKET BOT
<code>jokeapi.Request</code>	Your typed request struct	<code>TicketQuery{Status, Priority, Assignee, Since}</code>
<code>Schema()</code>	JSON schema generated from that struct	Enums for status and priority
<code>Normalize()</code>	Validation of model output	Reject unknown assignees, clamp date ranges
<code>get_joke</code> tool	One tool per API capability	<code>search_tickets</code> , <code>get_ticket</code> , <code>add_comment</code>
"No matching joke" as a tool result	Errors the model can act on	"0 tickets match. Try a wider date range."
Blacklist never loosened	Hard constraints in prompt <i>and</i> code	Read-only mode, no deleting, scope to one project
<code>seenJokes</code>	Exact state kept in code	Tickets already shown, IDs already modified
<code>trim()</code>	Context policy	Summarize old turns, pin the user's project and role
<code>-trace</code>	Observability	Log every prompt, tool call, result and token count

A decision guide



Start at the simplest stage that works. Each step down adds cost, latency and ways to fail.

Habits worth taking with you

Typed boundaries

Models write structs, not strings. Decode, normalize, then act.

Tool errors go to the model

Return failures as results so it can retry, instead of aborting the turn.

Cap every loop

Step limits, retry limits and output limits are cheap insurance.

Trace everything

Most of the learning, and most of the debugging, is reading the prompts and tool results.

Fake the model in tests

A scripted `httptest` server replays a conversation, so loop logic is tested without loading a model.

Measure with evals

Turn known failures into test cases before changing prompts.

Running it

```
ollama pull qwen3.5:9b-mlx
make test           # offline unit tests
make live           # also hits the real joke APIs
go run ./cmd/jokes -stage 3 -trace  # needs local Ollama
```

FLAG	DEFAULT	CONTROLS
-ctx	8192	Context window. Large defaults can push a 16 GB Mac into swap.
-keep-alive	2m	How long the model stays loaded when idle.
-think	off	Reasoning before acting, stages 2 and 3.
-trace	off	Print prompts, model output, tool calls and results to stderr.